

- 1) 1. O otimizador escolhe o S1 (força bruta) quando não existe nenhum índice criado sobre o atributo CPF, ou seja, o sistema não tem outra opção a não ser ler o arquivo inteiro bloco por bloco até encontrar o registro. Também pode acontecer de o otimizador preferir o S1 mesmo havendo índice, quando a tabela é muito pequena e o custo de percorrer o índice seria maior que simplesmente ler tudo de uma vez.

Já o S3a é escolhido quando existe um índice primário construído sobre o atributo CPF. Como o CPF é chave primária, a busca retorna no máximo um registro, e o índice primário permite chegar direto ao bloco do disco onde esse registro está, sem precisar ler o arquivo inteiro. O otimizador prefere esse caminho porque o custo é muito menor.

2. Para aplicar o S2 o arquivo físico da tabela FUNCIONARIO precisa estar ordenado no disco pelo atributo CPF. A pesquisa binária só funciona porque ela divide o arquivo ao meio repetidamente assumindo que os registros estão em ordem. Se o arquivo não estiver fisicamente ordenado por CPF esse método simplesmente não pode ser usado.

- 2) 1. A seletividade mede o quanto uma condição filtra os dados, ou seja, qual fração dos registros do arquivo ela descarta. Uma condição muito seletiva retorna poucos registros, o que é melhor para o desempenho. O otimizador estima a seletividade de cada condição separadamente usando estatísticas do catálogo do banco, como número de valores distintos e distribuição dos dados. No exemplo, se Dnr = 5 retorna 10% dos funcionários, Salario > 30000 retorna 40% e Sexo = F retorna 50%, o otimizador vai preferir começar pelo índice de Dnr porque ele é o mais seletivo e vai buscar o menor conjunto inicial de registros.

2. No S7 o otimizador escolhe apenas um dos índices disponíveis, o do atributo mais seletivo, para fazer o acesso inicial ao disco. Usando o índice de Dnr por exemplo, o sistema recupera todos os blocos que contêm funcionários do departamento 5 e os carrega na memória principal. Depois que esses registros já estão na memória, as outras condições (Salario > 30000 e Sexo = F) são verificadas diretamente ali, sem nenhum acesso adicional ao disco. Os registros que não satisfazem essas condições são simplesmente descartados na memória. O ponto chave é que o índice reduz a quantidade de blocos lidos do disco e o restante da filtragem acontece de graça na memória.

- 3) 1. O sort merge funciona em duas fases. Na fase de ordenação, o algoritmo divide o arquivo grande em pedaços menores que cabem na memória principal. Cada pedaço é carregado na memória, ordenado internamente por algum algoritmo como quicksort, e gravado de volta no disco como um arquivo temporário já ordenado. Esses arquivos temporários são chamados de runs. Na fase de intercalação o algoritmo pega vários runs ao mesmo tempo, lê o menor elemento da cabeça de cada um e vai mesclando tudo em um único arquivo ordenado. Se o número de runs for muito grande a intercalação pode acontecer em múltiplas rodadas, intercalando grupos de runs até sobrar apenas um arquivo final completamente ordenado.

2. O buffer na memória principal é essencial porque o disco é muito mais lento que a memória. Durante a fase de ordenação o buffer permite carregar um pedaço do arquivo, ordenar tudo rapidamente na memória e só então gravar de volta no disco. Durante a intercalação o buffer mantém um bloco de cada run sendo mesclado ao mesmo tempo, e um bloco de saída que vai sendo preenchido e gravado no disco quando enche. Quanto maior o buffer disponível, mais runs podem ser intercalados de uma vez e menos rodadas de intercalação são necessárias, o que reduz bastante o número total de leituras e gravações no disco.

- 4) 1. O racional é simples: seleção e projeção reduzem o tamanho dos dados. A seleção descarta linhas que não satisfazem a condição e a projeção descarta colunas desnecessárias. Se essas operações forem aplicadas antes de uma junção, as tabelas que entram na junção já são menores, o que reduz diretamente o custo da junção. A junção é a operação mais cara em termos de tempo e uso de disco, então qualquer coisa que diminua o tamanho das entradas antes dela é muito bem vinda. Aplicar seleção e projeção tarde demais significa carregar e processar dados que seriam descartados de qualquer jeito.

2. Quando a seleção é aplicada cedo o arquivo intermediário gerado para a próxima etapa tem menos registros. Isso significa que menos blocos precisam ser lidos e escritos no disco nas etapas seguintes. Se uma tabela com 1 milhão de registros é filtrada para 10 mil antes da junção, a junção opera sobre 10 mil linhas em vez de 1 milhão, o que pode reduzir o custo em várias ordens de grandeza. Além disso, arquivos intermediários menores ocupam menos espaço em disco e cabem mais facilmente na memória principal, o que também melhora o desempenho geral da consulta.

- 5) 1. Os principais componentes do custo de execução são o custo de acesso ao disco (leitura e escrita de blocos), o custo de processamento na CPU, o custo de uso da memória principal e em alguns casos o custo de comunicação em rede em bancos distribuídos. O acesso ao disco recebe maior ênfase porque a diferença de velocidade entre memória e disco é enorme, na ordem de milhares de vezes. Em bancos de dados grandes a maior parte do tempo de uma consulta é gasta esperando o disco transferir blocos para a memória, então minimizar o número de transferências de blocos é a forma mais eficaz de reduzir o tempo total de execução. Reduzir alguns milissegundos de CPU importa muito menos do que evitar a leitura de alguns milhares de blocos do disco.

2. Para a consulta $sDnumero > 5$ o método S4 usa o índice de ordenação sobre o atributo Dnumero para localizar o primeiro registro que satisfaz a condição, ou seja, o primeiro departamento com Dnumero igual a 6. Como o arquivo está fisicamente ordenado por Dnumero, todos os registros que satisfazem a condição estão armazenados de forma contígua no disco a partir desse ponto. O índice aponta diretamente para o bloco onde esse primeiro registro está. A partir daí o sistema simplesmente lê os blocos seguintes em sequência até o fim do arquivo, porque sabe que todos os registros restantes também satisfazem a condição $Dnumero > 5$. Isso evita leituras desnecessárias e aproveita a localidade física dos dados no disco.